LAB Report : System Integration & Validation

HW/SW Co-Design : FPGA vs STM32

Auteurs :
CADUSSEAU Rémi, JAROSKI Gustavo, MALAVAL Hugo, RIVAS Antoine

Professeur :
A. MASSOURI

Février 2026

Table des matières

1 Introduction : L'aboutissement du HW/SW Co-Design 2

2 Firmware Embarqué (STM32F746G-DISCO & Zephyr RTOS) 3
2.1 Architecture logicielle et Machine à états UART (FSM) 3
2.1.1 Le Contexte d'Interruption (ISR) : Capture et Décodage 3
2.1.2 La Transition ISR vers Thread : Le Sémaphore 4
2.2 Gestion de la Concurrence et Sécurité RTOS 4
2.3 Résilience du système : L'Horloge Virtuelle (Fallback) 5
2.3.1 Abstraction des requêtes temporelles 6
2.3.2 Le moteur de l'horloge logicielle (*Tick Engine*) 6
2.3.3 Notification bidirectionnelle de l'état 7

3 Logiciel Hôte et Interface de Contrôle (Python) 8
3.1 Architecture de l'Application et IHM (Double Dashboard) 8
3.1.1 Architecture Événementielle (Non-bloquante) 8
3.1.2 Le Double Dashboard (Monitoring Symétrique) 8
3.1.3 Commandes Unifiées et Synchronisation Absolue 9
3.2 Asynchronisme, Auto-Refresh et Watchdog matériel 9
3.2.1 L'Auto-Refresh : Le cœur battant du système 10
3.2.2 Résilience : Le Watchdog Matériel (Surveillance des Ports COM) 10
3.3 Outil d'Acquisition de Données (Data Logger CSV) 11
3.3.1 Chronométrage et Profilage 11
3.3.2 Exportation et Formatage pour Tableur 12
3.3.3 Génération de la Courbe de Tendance 12

4 Validation Système et Analyse Comparative (HW vs SW) 13
4.1 Validation Fonctionnelle Multi-Plateformes 13
4.1.1 Matrice de Validation des Trames 13
4.1.2 Le Test Critique : L'événement Asynchrone Simultané 13
4.2 Analyse de la Latence et de la Dérive (Hardware vs Software) 14
4.2.1 Explication du phénomène en « Dents de Scie » (Aliasing) 14
4.2.2 Le Surcoût Logiciel : Analyse de la ligne de base 14

5 Conclusion Générale du Projet 16

1 Introduction : L'aboutissement du HW/SW Co-Design

Ce rapport marque l'aboutissement de notre projet global de *Hardware/Software Co-Design*. Tout au long des phases précédentes, nous avons conçu, simulé et implémenté une architecture matérielle complète sur FPGA (Nexys A7 100T). Cette conception comprenait un cœur d'horloge temps réel (RTC), une couche de communication asynchrone robuste (UART Transceiver), ainsi qu'une *User Logic* agissant comme un routeur transactionnel intelligent. Le FPGA est ainsi devenu un composant matériel autonome, déterministe et pilotable à distance.

Cependant, l'ingénierie des systèmes embarqués modernes repose rarement sur des solutions purement matérielles. L'enjeu de cette ultime étape est de concevoir le pendant « Logiciel » (*Software*) de notre système, de développer l'interface maître sur PC, et de confronter ces deux paradigmes.

Pour ce faire, notre travail s'articule autour de trois axes majeurs qui structureront ce document :

- **Le Firmware Embarqué (STM32 & Zephyr) :** Nous avons programmé un microcontrôleur industriel (STM32F746G-DISCO) opérant sous le système d'exploitation temps réel Zephyr (RTOS). Son rôle est de piloter un circuit RTC externe (PCF8563) via un bus I^2C, tout en respectant scrupuleusement le même protocole de communication UART que celui défini pour le FPGA.
- **Le Logiciel Hôte (Python) :** Afin d'orchestrer ce système hybride, nous avons développé une application maître en Python. Celle-ci intègre un tableau de bord bidirectionnel, une gestion avancée de l'asynchronisme (Auto-Refresh, Watchdog série), ainsi qu'un outil d'acquisition de données (Data Logger CSV) destiné à profiler les performances temporelles.
- **L'Analyse Comparative (HW vs SW) :** L'objectif suprême de ce projet est d'aligner ces deux architectures sur la même ligne de départ. En synchronisant le FPGA et le STM32 à la milliseconde près et en provoquant des événements simultanés, nous mettrons en évidence les différences fondamentales entre l'exécution « Cycle-Accurate » du silicium et la latence intrinsèque (surcoût) introduite par un RTOS et des bus de communication sériels.

Ce document détaille l'implémentation de ces couches logicielles, expose les défis liés à la concurrence des tâches, et conclut par une analyse chiffrée de la dérive temporelle entre le monde matériel et le monde logiciel.

2 Firmware Embarqué (STM32F746G-DISCO & Zephyr RTOS)

2.1 Architecture logicielle et Machine à états UART (FSM)

Contrairement à l'approche matérielle sur FPGA où les modules (UART, RTC, Décodeur) s'exécutent en parallèle grâce à la nature spatiale du silicium, le STM32 repose sur une exécution séquentielle orchestrée par un unique cœur de processeur (Cortex-M7). L'enjeu majeur de cette implémentation logicielle est de garantir la réactivité du système face aux trames asynchrones du PC, sans pour autant bloquer les autres tâches critiques (rafraîchissement de l'écran, gestion de l'horloge interne, communication I^2C).

Pour répondre à cette contrainte, nous avons conçu une architecture scindée en deux contextes d'exécution distincts, reliés par une primitive de synchronisation Zephyr : le **Sémaphore**.

2.1.1 Le Contexte d'Interruption (ISR) : Capture et Décodage

La réception des octets ne se fait pas par scrutation active (*polling*), ce qui monopoliserait le processeur. Nous avons configuré le driver UART de Zephyr en mode *Interrupt-Driven*. À chaque octet reçu, le microcontrôleur suspend brièvement son activité courante pour exécuter une routine de service d'interruption (ISR) : la fonction `uart_cb()`.

Au sein de cette ISR, une Machine à États Finis (FSM) logicielle, calquée sur le comportement de notre *User Logic* VHDL, traite l'octet à la volée.

```c
void process_uart_rx(uint8_t byte) {
    switch (rx_state) {
        case IDLE:
            if (byte == SOF_BYTE) rx_state = READ_CMD;
            break;
        case READ_CMD:
            rx_cmd = byte;
            calc_crc = byte;
            rx_state = READ_LEN;
            break;
        case READ_LEN:
            rx_len = byte;
            calc_crc ^= byte;
            rx_payload_cnt = 0;
            rx_state = (rx_len == 0) ? READ_CRC : READ_PAYLOAD;
            break;
        case READ_PAYLOAD:
            if (rx_payload_cnt < sizeof(rx_payload))
                rx_payload[rx_payload_cnt] = byte;
            rx_payload_cnt++;
            calc_crc ^= byte;
            if (rx_payload_cnt >= rx_len) rx_state = READ_CRC;
            break;
        case READ_CRC:
            if (byte == calc_crc) k_sem_give(&cmd_sem); // Validation !
            else                  send_nack();          // Erreur
            rx_state = IDLE;
            break;
    }
}
```

Listing 1 – Extrait C : FSM logicielle exécutée sous contexte d'interruption

Ce code illustre la frugalité requise pour une ISR : aucune boucle bloquante, aucune attente. Le calcul du CRC (`calc_crc ^= byte`) est réalisé dynamiquement à chaque état.

2.1.2 La Transition ISR vers Thread : Le Sémaphore

La règle d'or d'un OS Temps Réel (RTOS) stipule qu'une ISR doit être la plus courte possible. Ainsi, lorsque la FSM atteint l'état `READ_CRC` et confirme l'intégrité de la trame, elle **n'exécute pas** la commande (qui requiert des accès lents sur le bus I^{2}C). À la place, elle invoque `k_sem_give(&cmd_sem)`.

Cette instruction agit comme un jeton. Elle libère instantanément le *Worker Thread* principal qui était endormi en attente de ce sémaphore via l'instruction `k_sem_take(&cmd_sem, K_MSEC(10))`.

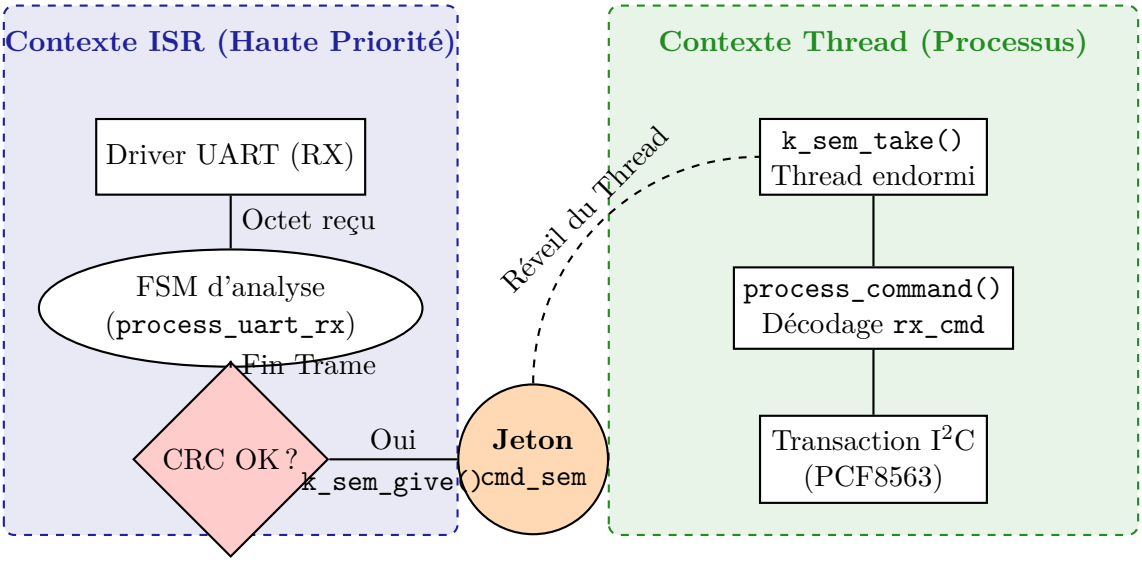

Figure 1 – Architecture logicielle de décodage UART sous Zephyr RTOS. Séparation stricte entre le contexte d'interruption asynchrone (gauche) et le thread d'exécution synchrone (droite) via un Sémaphore.

Cette séparation des privilèges offre au STM32 un comportement asynchrone bluffant, imitant presque le parallélisme matériel du FPGA. Pendant que le *Worker Thread* est occupé à écrire l'heure dans la puce RTC via l'I^{2}C, l'ISR UART reste armée et peut déjà commencer à réceptionner et analyser les octets de la commande suivante en tâche de fond.

2.2 Gestion de la Concurrence et Sécurité RTOS

L'introduction d'un système d'exploitation temps réel (RTOS) et d'une interface graphique (LVGL) complexifie drastiquement la gestion de la mémoire. Contrairement au FPGA où chaque registre dispose de son propre chemin de données physique, le STM32 partage sa mémoire (RAM) entre plusieurs tâches concurrentes : la réception UART (sous interruption), le traitement des commandes, l'incrémentation de l'horloge logicielle et le rafraîchissement de l'écran.

Pour garantir la sécurité transactionnelle et éviter les conditions de course (*Race Conditions*), notre architecture repose sur trois mécanismes de synchronisation stricts offerts par le noyau Zephyr :

- **L'Exclusion Mutuelle (Mutex) :** L'état de l'alarme (`al_on`, `al_hour`, etc.) peut être modifié à la fois par l'UART (commande `0x04`) et par l'utilisateur via l'écran tactile. Un `k_mutex` (`alarm_mutex`) protège ces variables. Toute lecture ou écriture nécessite la capture préalable de ce mutex, garantissant que l'état n'est jamais altéré en cours d'évaluation.
- **Les Variables Atomiques (`atomic_t`) :** Pour notifier le *Thread* principal qu'une activité RX/TX a eu lieu (afin de faire clignoter les LEDs virtuelles à l'écran), l'utilisation de simples booléens `volatile` est insuffisante face aux interruptions. Nous avons utilisé des

variables atomiques avec l'instruction `atomic_cas()` (*Compare-And-Swap*), garantissant que la lecture et la réinitialisation du drapeau s'effectuent en un seul cycle CPU indivisible.

— **Les Files de Messages (`k_msgq`) :** Pour transférer l'heure courante du *Worker Thread* vers la tâche d'affichage LVGL, une file de messages est utilisée. Cela évite le phénomène de lecture corrompue (*Torn Read*), où l'interface graphique pourrait lire les secondes de la minute courante, mais les heures de la minute précédente. Zephyr garantit la copie intégrale et sécurisée de la structure `ui_time_msg`.

ANALYSE CRITIQUE : L'Empreinte Mémoire (Profilage)
Contexte : À l'issue de la compilation, l'éditeur de liens de Zephyr a généré le profilage mémoire de notre firmware. L'application consomme **350 Ko de mémoire FLASH (33.3%)** et **120 Ko de RAM statique (45.9%)**.
Comparaison HW/SW : Ces métriques illustrent parfaitement le coût de l'abstraction logicielle. Là où notre *User Logic* matérielle sur FPGA nécessitait moins de 1% des ressources logiques sans recourir à la moindre mémoire bloc (BRAM), l'approche logicielle impose d'embarquer un noyau OS complet, un ordonnanceur de tâches et un *framebuffer* graphique, justifiant cette lourde empreinte mémoire.

2.3 Résilience du système : L'Horloge Virtuelle (Fallback)

Dans un contexte de système embarqué industriel, la perte d'un périphérique matériel (défaillance du bus I^2C, câble débranché, puce endommagée) ne doit en aucun cas entraîner un effondrement critique du système (*System Crash*) ou un blocage de la communication série.

Pour garantir cette haute disponibilité, nous avons doté notre firmware d'un mécanisme de **Virtualisation (Fallback automatique)**. L'idée est simple : si le composant physique PCF8563 ne répond pas au démarrage, le microcontrôleur STM32 bascule silencieusement sur une horloge logicielle (émulée en RAM) tout en continuant à répondre aux requêtes UART du PC exactement comme si de rien n'était.

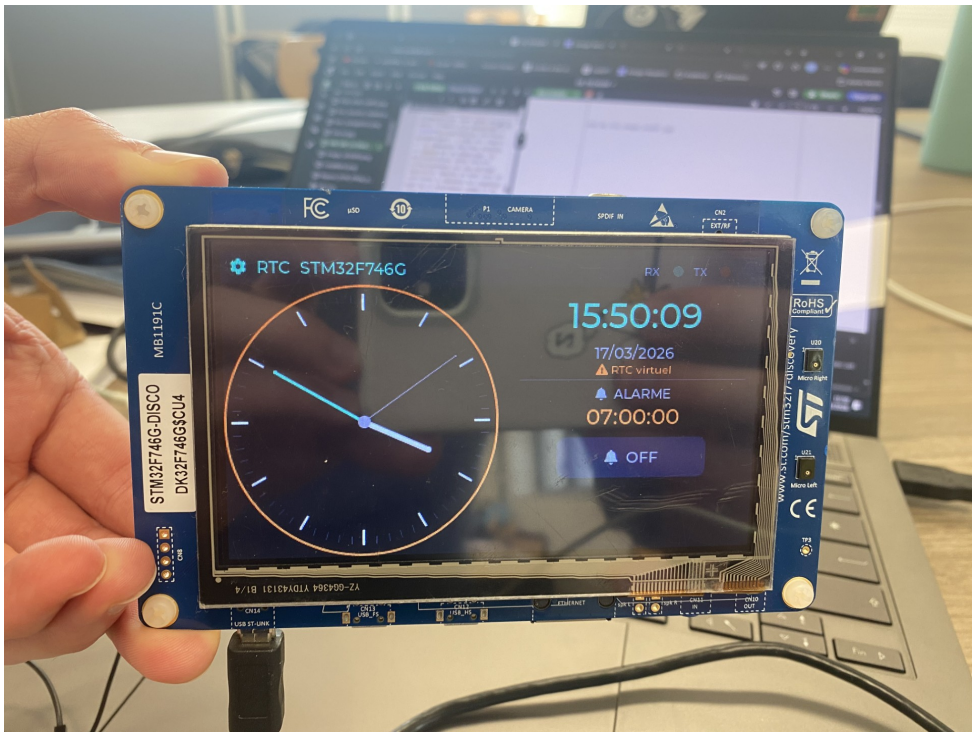

Figure 2 – Carte STM32F746G-DISCO utilisée pour l'implémentation du firmware sous Zephyr RTOS.

2.3.1 Abstraction des requêtes temporelles

Pour que la machine à états UART (FSM) et l'interface graphique (LVGL) n'aient pas à se soucier de la source physique du temps, nous avons créé une couche d'abstraction via deux fonctions maîtresses : `rtc_get_virtual()` et `rtc_set_virtual()`.

```c
/* Lecture unifiee : RTC physique si dispo, sinon sw_clock (RAM) */
static int rtc_get_virtual(struct rtc_time *tm)
{
    if (rtc_available) {
        return rtc_get_time(rtc_dev, tm); // Acces materiel I2C
    }
    // Securite multi-threads pour l'acces en RAM
    k_mutex_lock(&sw_clock_mutex, K_FOREVER);
    *tm = sw_clock;
    k_mutex_unlock(&sw_clock_mutex);
    return 0;
}
```

Listing 2 – Extrait C : Couche d'abstraction et routage de l'horloge

Au démarrage, le système teste la présence du RTC via la macro Zephyr `device_is_ready(rtc_dev)`. Le booléen global `rtc_available` conditionne ensuite tout le routage du système.

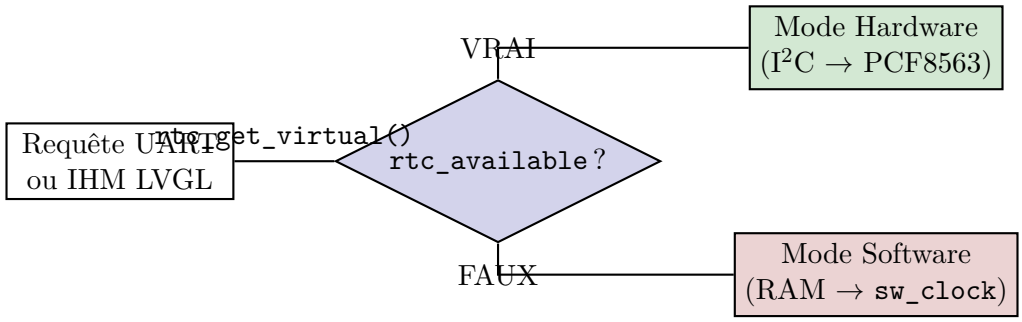

Figure 3 – Mécanisme de routage (Fallback) abstrait garantissant la tolérance aux pannes matérielles.

2.3.2 Le moteur de l'horloge logicielle (*Tick Engine*)

Lorsque le mode virtuel est actif, le temps doit tout de même s'écouler. Ce rôle est délégué au *Worker Thread* qui, indépendamment de ses autres tâches, exécute la fonction `sw_clock_tick()` toutes les 1000 millisecondes.

Cette fonction simule le comportement matériel de la RTC en gérant l'incrémentation des secondes, la retenue des minutes, des heures, et le débordement complexe des mois (via un tableau de constantes `days_in_month[12]`). Pour éviter toute lecture corrompue pendant l'incrémentation de la structure, cette opération critique est intégralement encapsulée par le `sw_clock_mutex`.

```c
/* Incremente sw_clock d'une seconde (appele par le worker toutes les 1000 ms)
    */
static void sw_clock_tick(void)
{
    k_mutex_lock(&sw_clock_mutex, K_FOREVER);
    sw_clock.tm_sec++;
    if (sw_clock.tm_sec >= 60) {
        sw_clock.tm_sec = 0;
        sw_clock.tm_min++;
        // ... Logique de retenue jusqu'aux annees ...
    }
    k_mutex_unlock(&sw_clock_mutex);
}
```

Listing 3 – Extrait C : Incrémentation protégée de l'horloge virtuelle

2.3.3 Notification bidirectionnelle de l'état

Afin d'offrir une transparence totale à l'utilisateur, ce mode dégradé est signalé sur l'ensemble des interfaces du système :

1. **Sur l'écran physique du STM32 (LVGL) :** L'interface change la couleur du statut et de l'aiguille des secondes (passant du orange au gris) et affiche l'alerte « RTC Virtuel ».
2. **Sur l'application Python Hôte :** La FSM du STM32 code cette information dans le Bit 1 du registre de statut (réponse à la commande `0x07 GET STATUS`).

```
case CMD_GET_STATUS:
{
    k_mutex_lock(&alarm_mutex, K_FOREVER);
    uint8_t local_on = al_on;
    k_mutex_unlock(&alarm_mutex);

    // Bit 0 = Etat de l'alarme, Bit 1 = Mode RTC (0 = Physique, 1 = Virtuel)
    uint8_t status = local_on | (rtc_available ? 0x00 : 0x02);
    crc = CMD_GET_STATUS ^ 0x01 ^ status;

    send_byte(SOF_BYTE); send_byte(CMD_GET_STATUS); send_byte(0x01);
    send_byte(status); send_byte(crc);
}
break;
```

Listing 4 – Extrait C : Codage de la disponibilité RTC dans le Status Byte

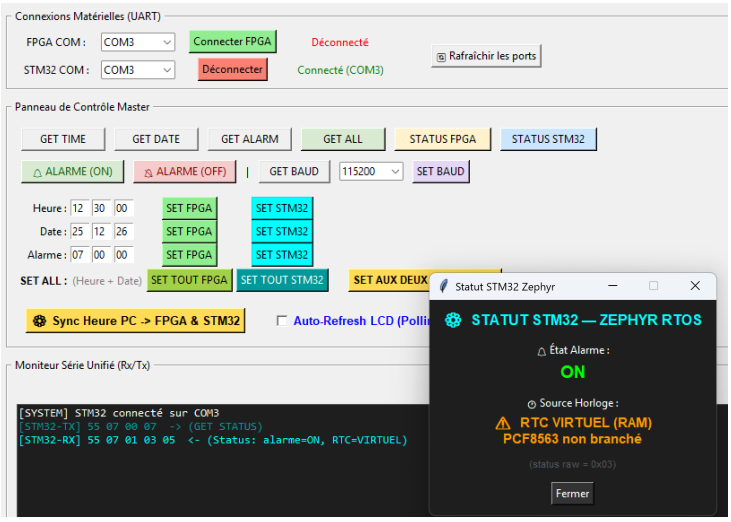

Figure 4 – Notification de l'état de l'horloge sur l'application hôte Python. Le logiciel décode le bit 1 du *Status Byte* pour alerter l'utilisateur du mode dégradé.

Cette résilience démontre l'intérêt du développement sous RTOS : la séparation des tâches permet de simuler un comportement matériel complexe (horloge) sans perturber le cœur de communication du système.

3 Logiciel Hôte et Interface de Contrôle (Python)

Le dernier pilier de notre architecture de *Hardware/Software Co-Design* est l'application maître exécutée sur le PC hôte. Développée en Python, elle s'interface simultanément avec nos deux systèmes embarqués (FPGA et STM32) via le protocole UART pour les configurer, les monitorer et comparer leurs performances en temps réel.

3.1 Architecture de l'Application et IHM (Double Dashboard)

L'application repose sur la bibliothèque graphique `Tkinter` pour l'Interface Homme-Machine (IHM) et sur `pyserial` pour la gestion de la couche physique (USB/UART). L'ensemble du système est encapsulé dans une classe orientée objet `RTC_App`.

3.1.1 Architecture Événementielle (Non-bloquante)

Piloter deux ports séries (COM) distincts tout en maintenant une interface graphique fluide est un défi classique. L'utilisation de *Threads* multiples en Python (soumis aux contraintes du GIL - *Global Interpreter Lock*) peut s'avérer lourde et complexe à synchroniser avec l'IHM.

Nous avons donc opté pour une architecture **purement événementielle**. La scrutation des buffers matériels (RX) et la mise à jour des horloges sont déléguées à des tâches périodiques asynchrones via la méthode `root.after()` de Tkinter.

```python
class RTC_App:
    def __init__(self, root):
        # ... [Initialisation de l'IHM et des ports] ...

        # Lancement des boucles d'ecoute non-bloquantes (Polling)
        self.root.after(50,  self.listen_fpga)    # Scrutation RX FPGA toutes les 50ms
        self.root.after(50,  self.listen_stm32)   # Scrutation RX STM32 toutes les 50ms
        self.root.after(500, self.auto_refresh_task) # Requete de l'heure toutes les 500ms

        # Watchdog port : verifie toutes les 2 s si les ports sont encore vivants
        self.root.after(2000, self.watchdog_ports)
```

Listing 5 – Extrait Python : Ordonnancement asynchrone des tâches dans le constructeur de l'application

Cette approche garantit que l'application "avale" le flux de données série sans jamais figer l'interface graphique de l'utilisateur, tout en maintenant une réactivité de l'ordre de 50 millisecondes.

3.1.2 Le Double Dashboard (Monitoring Symétrique)

Afin de mettre en exergue la comparaison matérielle/logicielle, l'IHM a été structurée autour d'un tableau de bord visuel strictement symétrique :

- **Le Panneau FPGA (Vert) :** Dédié à la carte Nexys A7. Il affiche le décodage en temps réel des trames renvoyées par notre *User Logic* VHDL.
- **Le Panneau STM32 (Cyan) :** Dédié au microcontrôleur sous Zephyr RTOS. Il intègre un indicateur dynamique signalant si le composant I^2C (PCF8563) est physiquement reconnu ou si la carte opère sur son horloge virtuelle de secours (RAM).

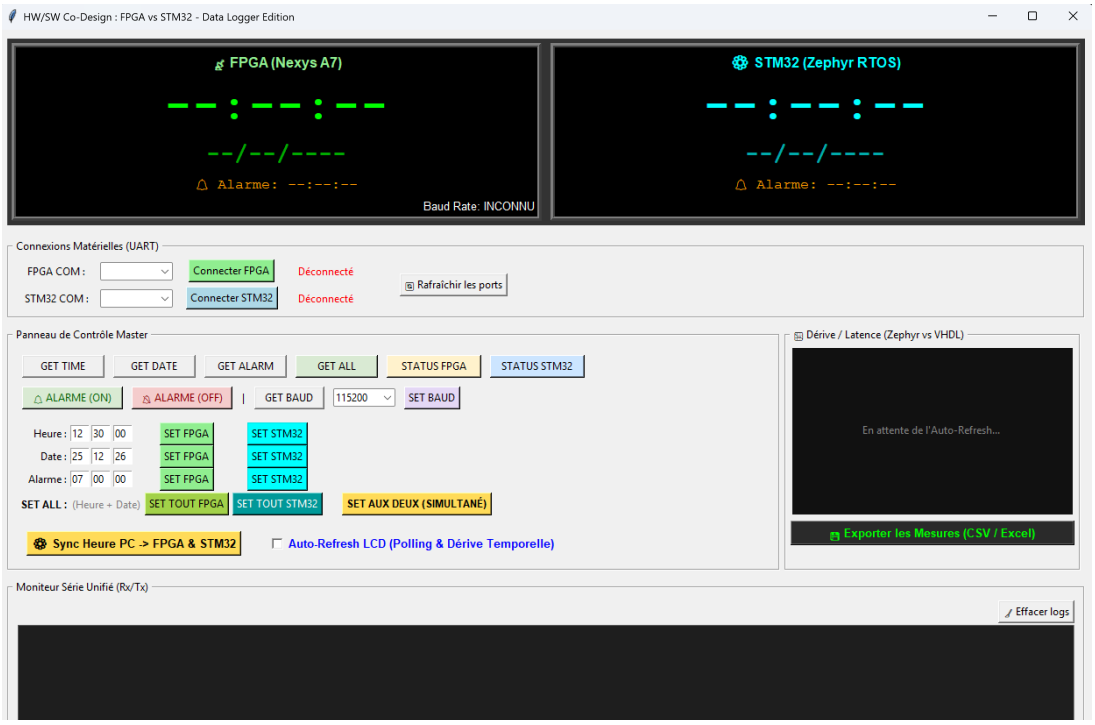

Figure 5 – Interface Homme-Machine (IHM) Python : Le *Double Dashboard* permet le monitoring simultané des registres VHDL et des variables du RTOS.

3.1.3 Commandes Unifiées et Synchronisation Absolue

Situé sous les écrans de monitoring, un panneau de contrôle unifié centralise la génération des trames UART sortantes. Il offre une granularité totale : l'utilisateur peut cibler une seule carte ou déclencher des événements parfaitement synchronisés sur les deux systèmes.

L'exemple le plus représentatif de la puissance de ce système de contrôle est la fonction de synchronisation sur l'horloge du PC hôte :

```
    def sync_pc_time(self):
        # 1. Capture du timestamp absolu de Windows/Linux
        now = datetime.datetime.now()

        # 2. Formatage du Payload (Mise a l'echelle pour le BCD materiel)
        payload = [now.second, now.minute, now.hour,
                   now.day, now.month, now.year % 100]

        # 3. Diffusion simultanee de la trame SET ALL (0x02)
        self.send_fpga(CMD_SET_ALL, payload)
        self.send_stm32(CMD_SET_ALL, payload)

        self.log("[SYSTEM]␣      ␣Sync␣PC␣->␣DEUX␣CARTES␣envoyee␣!", "sys")
        self.reset_drift_graph()
```

Listing 6 – Extrait Python : Synchronisation absolue des deux cartes sur l'horloge de l'OS hôte

Cette méthode garantit que le composant matériel (FPGA) et le composant logiciel (STM32) démarrent strictement à la même seconde, posant ainsi une base saine et équitable pour nos mesures comparatives de dérive temporelle.

3.2 Asynchronisme, Auto-Refresh et Watchdog matériel

Pour répondre aux exigences du cahier des charges stipulant que le logiciel doit interroger périodiquement les cartes et détecter les alarmes de manière transparente, nous avons implémenté

des mécanismes de tâches de fond (*Background Tasks*) gérant l'asynchronisme et la résilience matérielle.

3.2.1 L'Auto-Refresh : Le cœur battant du système

La mise à jour de l'affichage n'est pas passive. Une tâche planifiée, `auto_refresh_task()`, s'exécute toutes les 500 millisecondes. Elle agit comme le métronome de l'application en interrogeant simultanément les deux systèmes embarqués.

```python
    def auto_refresh_task(self):
        if self.var_autorefresh.get():
            # Interrogation du FPGA
            if self.fpga_port and self.fpga_port.is_open:
                self.t_last_req_fpga = time.time() # Lancement du chrono de latence
                self.send_fpga(CMD_GET_TIME, [])
                self.send_fpga(CMD_GET_STATUS, [])

            # Interrogation du STM32
            if self.stm32_port and self.stm32_port.is_open:
                self.t_last_req_stm32 = time.time()
                self.send_stm32(CMD_GET_TIME, [])

        self.root.after(500, self.auto_refresh_task) # Replanification
```

Listing 7 – Extrait Python : La tâche d'interrogation périodique (Polling)

Le déclenchement du chronomètre absolu (`time.time()`) juste avant l'envoi de la commande `CMD_GET_TIME` est la clé de voûte de notre système de profilage. Il permet de mesurer avec une précision de l'ordre de la milliseconde le temps exact d'aller-retour (*Round-Trip Time*) de chaque requête, incluant le traitement par la couche UART matérielle et l'OS Zephyr.

3.2.2 Résilience : Le Watchdog Matériel (Surveillance des Ports COM)

L'un des défis majeurs rencontrés lors du développement informatique lié à l'embarqué (particulièrement lors de l'utilisation de Machines Virtuelles Linux sous hôte Windows) est la perte inopinée du port série. Un câble USB effleuré ou une réattribution automatique du périphérique par l'hyperviseur provoque instantanément une erreur fatale (`OSError` ou `PermissionError`) faisant crasher l'application.

Pour pallier cette fragilité, nous avons conçu un **Watchdog Matériel** (Chien de garde) logiciel. Exécutée toutes les 2 secondes, cette routine s'assure que le système d'exploitation possède toujours les droits d'accès physiques aux ports COM.

```python
    def watchdog_ports(self):
        """ Tente une lecture CTS (0 octet) sur chaque port ouvert. """
        if self.fpga_port and self.fpga_port.is_open:
            try:
                # getCTS() leve une exception si le port n'est plus accessible physiquement
                self.fpga_port.getCTS()
            except (serial.SerialException, OSError) as e:
                self.log(f"[WATCHDOG]  Port FPGA perdu : {e}", "warn")
                self._force_disconnect_fpga() # Fermeture propre
```

Listing 8 – Extrait Python : Le Watchdog série utilisant le signal matériel CTS

La subtilité architecturale réside dans l'appel à la fonction `getCTS()` (*Clear To Send*). Cette instruction demande au driver USB d'interroger l'état d'une broche matérielle du convertisseur FTDI sans pour autant vider le buffer de données (RX). Si la requête échoue, l'exception est interceptée (*try/except*), le port est déclaré "Perdu", et le logiciel bascule dans un mode

de tentative de **reconnexion automatique** (`_try_reconnect_fpga`) toutes les 3 secondes, jusqu'au retour du périphérique.

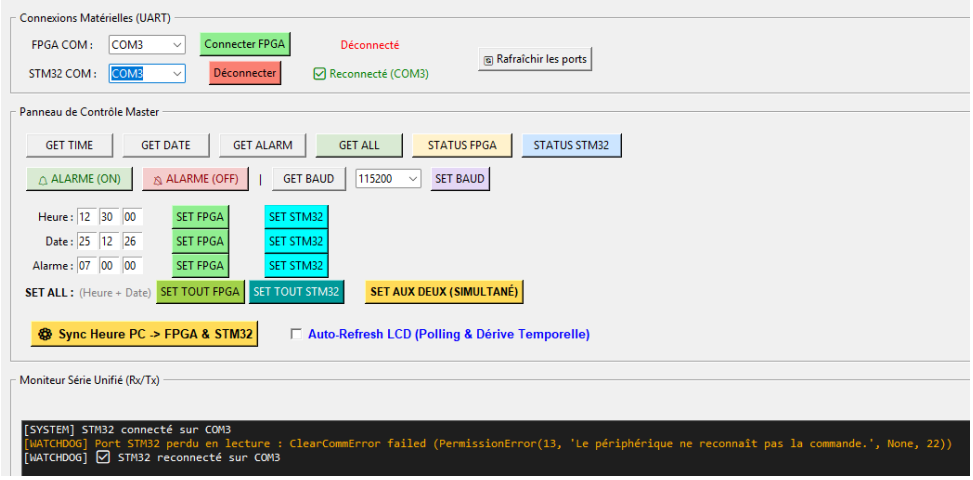

Figure 6 – Action du Watchdog matériel : Le logiciel intercepte la déconnexion physique brutale du port USB, avertit l'utilisateur et initialise un processus de reconnexion automatique sans interrompre l'exécution globale.

Cette couche de sécurité transforme une interface de débogage classique en une solution logicielle robuste, digne d'un environnement de test industriel où la fiabilité de la connexion hôte-cible ne peut être garantie à 100%.

3.3 Outil d'Acquisition de Données (Data Logger CSV)

Afin de répondre à l'exigence d'analyse comparative des performances (Section 8 du cahier des charges : *« Measure and log timing differences »*), la simple observation visuelle des compteurs sur le tableau de bord n'est pas suffisante. Il a fallu doter l'application Python d'un moteur d'acquisition de données (*Data Logger*) capable de profiler le système à la milliseconde près.

3.3.1 Chronométrage et Profilage

Lorsqu'une salve d'interrogation est déclenchée par l'Auto-Refresh, le système enregistre le *Timestamp* exact du départ de la requête. Dès que la trame `0x11 GET TIME` est acquittée et décodée par l'une des cartes, la latence matérielle est calculée.

```python
    def check_and_update_graph(self):
        if self.lat_fpga is not None and self.lat_stm32 is not None:
            # Calcul du surcout logiciel du RTOS
            delta_ms = self.lat_stm32 - self.lat_fpga

            # Sauvegarde de l'echantillon temporel absolu (en secondes)
            elapsed = round(time.time() - self.t_start, 3)

            # Ajout au Data Logger
            self.full_log_data.append([
                elapsed,
                round(self.lat_fpga, 2),
                round(self.lat_stm32, 2),
                round(delta_ms, 2)
            ])
```

Listing 9 – Extrait Python : Enregistrement de la latence dans le buffer d'historique

Ce tableau bidimensionnel (`self.full_log_data`) s'enrichit dynamiquement en tâche de fond de milliers de points de mesure, capturant ainsi l'évolution de la dérive temporelle sans perturber le fil d'exécution principal.

3.3.2 Exportation et Formatage pour Tableur

Pour permettre une exploitation scientifique de ces données, une fonction d'exportation au format CSV a été intégrée. Une attention particulière a été portée au formatage des données pour garantir une compatibilité native avec les tableurs (comme Microsoft Excel) configurés selon les standards régionaux européens.

```python
    def export_csv(self):
        # ... [Ouverture du fichier via boite de dialogue] ...
        with open(filepath, mode='w', newline='') as f:
            # Utilisation du point-virgule comme separateur de colonnes
            writer = csv.writer(f, delimiter=';')
            writer.writerow(["Temps Ecoule (s)", "Latence FPGA (ms)",
                             "Latence STM32 (ms)", "Surcout RTOS (ms)"])

            for row in self.full_log_data:
                # Remplacement des points par des virgules pour compatibilite Excel
                formatted_row = [str(v).replace('.', ',') for v in row]
                writer.writerow(formatted_row)
```

Listing 10 – Extrait Python : Sérialisation CSV avec formatage régional (Virgule flottante)

3.3.3 Génération de la Courbe de Tendance

Grâce à cet outil d'export, nous avons pu capturer le comportement du système sur une longue période de *Polling*. Les données générées dans le fichier `Report_Drift_FPGA_STM32.csv` ont ensuite été traitées pour extraire le graphique d'oscilloscope suivant :

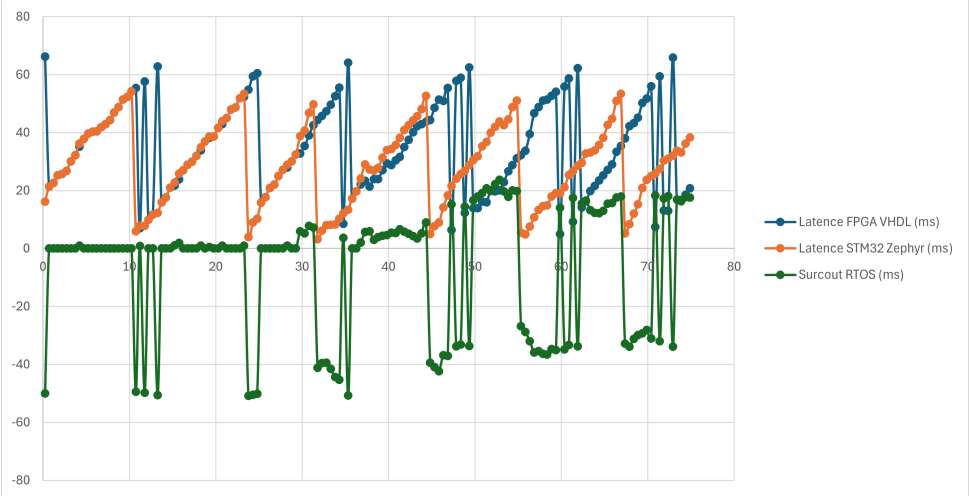

Figure 7 – Profilage temporel généré à partir de l'export CSV du Data Logger. En bleu la latence du FPGA (VHDL), en orange celle du STM32 (Zephyr), et en vert la différence nette représentant le surcoût du RTOS.

L'interprétation technique de ce graphique atypique (en dents de scie) et l'analyse chiffrée de la différence entre le monde matériel et logiciel feront l'objet du chapitre suivant dédié à la validation du système.

4 Validation Système et Analyse Comparative (HW vs SW)

L'objectif final de notre démarche de *Hardware/Software Co-Design* est d'évaluer les mérites respectifs d'une implémentation matérielle pure (VHDL sur FPGA) face à une implémentation logicielle embarquée (C/Zephyr sur STM32).

Cependant, avant de profiler les performances temporelles du système, il est impératif de s'assurer de **l'équivalence fonctionnelle** absolue des deux plateformes. Le FPGA et le STM32 doivent réagir de manière strictement identique aux mêmes sollicitations du protocole UART.

4.1 Validation Fonctionnelle Multi-Plateformes

Pour garantir cette équivalence, nous avons soumis les deux cibles matérielles à un plan de test commun (Cahier de Recette) piloté depuis notre application Python.

4.1.1 Matrice de Validation des Trames

Le tableau ci-dessous synthétise les résultats des tests fonctionnels menés simultanément sur les deux cartes. Chaque commande a été injectée sur le bus série et la réaction de la cible a été observée.

ID Commande	Description du Test	Résultat FPGA	Résultat STM32
`0x01 / 0x11`	Lecture de l'heure et de la date (GET)	**Validé**	**Validé**
`0x02 / 0x12`	Écriture de l'heure et de la date (SET)	**Validé**	**Validé**
`0x0B`	Forçage de l'état de l'alarme (TOGGLE)	**Validé**	**Validé**
`0x06`	Changement dynamique du Baud Rate	**Validé**	**Validé**
`N/A`	Injection volontaire d'un CRC erroné	**Rejet (NACK)**	**Rejet (NACK)**

Table 1 – Matrice de validation fonctionnelle prouvant l'équivalence comportementale des implémentations HW et SW face au même protocole de communication.

4.1.2 Le Test Critique : L'événement Asynchrone Simultané

La validation la plus complexe de notre architecture consistait à prouver que le logiciel Python était capable de gérer l'asynchronisme parfait des deux plateformes. Le test s'est déroulé selon le scénario suivant :

1. **Synchronisation :** Envoi d'une trame `SET ALL` simultanée pour aligner l'horloge du FPGA et du STM32 à la seconde près.
2. **Armement :** Configuration d'une alarme sur les deux cartes pour qu'elle sonne exactement 15 secondes plus tard.
3. **Déclenchement :** Arrivée à l'heure cible, les deux cartes doivent prendre l'initiative de la communication et envoyer la trame `0x0A ALARM EVENT` vers le PC.

Le test fut un succès total. Comme l'illustre le journal d'événements (*Log Terminal*) de notre application, le logiciel hôte a reçu les deux trames `0x0A` dans la même fraction de seconde. Le *Thread* principal de l'application Python a correctement intercepté ces deux événements asynchrones, déclenchant instantanément l'animation visuelle sur l'interface et jouant les deux signaux sonores distincts (Aigu pour le FPGA, Grave pour le STM32) via la librairie `winsound`.

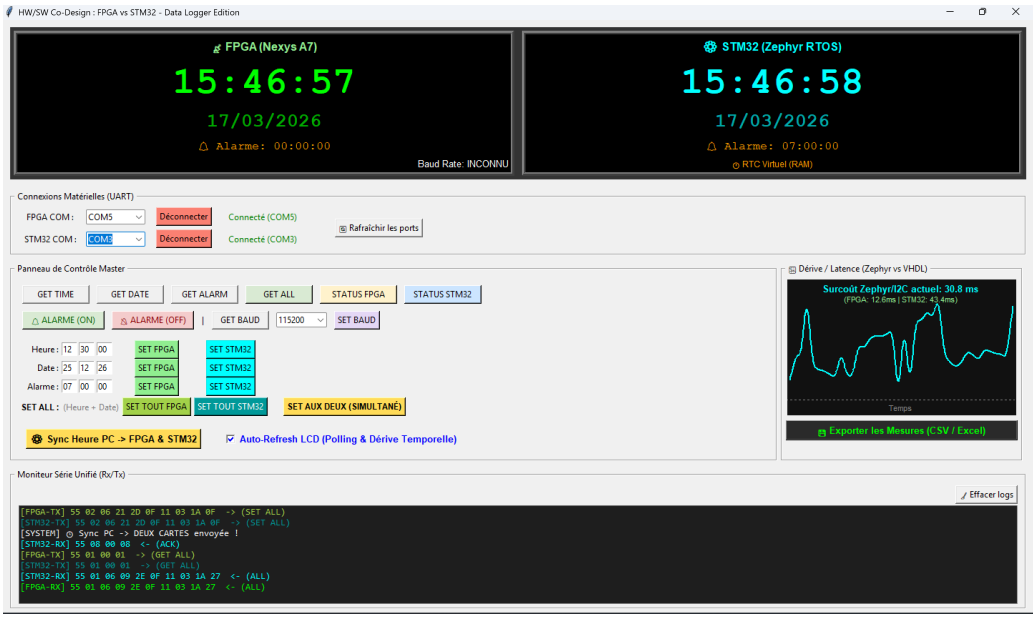

Figure 8 – Extrait du moniteur série unifié confirmant la réception parfaitement simultanée des trames d'alarme asynchrones provenant du FPGA et du STM32.

Cette validation fonctionnelle absolue nous autorise désormais à passer à l'étape finale du projet : l'étude profilée des différences de latence et de dérive temporelle entre le silicium et le système d'exploitation.

4.2 Analyse de la Latence et de la Dérive (Hardware vs Software)

Pour évaluer objectivement les performances des deux architectures, nous avons utilisé l'outil d'acquisition de données (Data Logger) décrit au Chapitre 3. Le système hôte a envoyé une requête `GET TIME` simultanée toutes les 500 ms aux deux cartes sur une période prolongée, et a mesuré le temps d'aller-retour (*Round-Trip Time*).

Le graphique généré (Figure 7) présente l'évolution temporelle de ces latences. Son allure atypique révèle deux phénomènes distincts qu'il convient d'analyser séparément : un artefact de mesure lié au PC, et la véritable différence de performance entre le FPGA et le STM32.

4.2.1 Explication du phénomène en « Dents de Scie » (Aliasing)

La première observation frappante est la forme en dents de scie parfaitement périodique des courbes bleue (FPGA) et orange (STM32). Ce comportement **n'est pas** un défaut des cartes embarquées, mais un artefact métrologique induit par le système d'exploitation du PC hôte (Windows/Linux).

L'application Python utilise la fonction `root.after(500)` pour cadencer ses requêtes. Cependant, un OS de bureau n'est pas "Temps Réel". La boucle prend en réalité un peu plus de 500 ms à s'exécuter. Ce léger décalage s'accumule linéairement à chaque cycle : le PC "écoute" la réponse un peu plus tard à chaque fois, ce qui augmente artificiellement la latence mesurée. Une fois que ce décalage cumulé atteint la durée d'un cycle de traitement interne de l'application hôte (environ 50 à 60 ms), le buffer de réception rattrape son retard, et la latence mesurée chute brutalement à son niveau de base, créant ce motif de scie (phénomène de battement ou *Aliasing*).

4.2.2 Le Surcoût Logiciel : Analyse de la ligne de base

Pour extraire la véritable latence intrinsèque de nos deux systèmes embarqués, il faut lire le graphique sur les points les plus bas des dents de scie (la ligne de base, là où l'artefact du PC est

nul).

Sur ces points (par exemple autour de $t = 10s$ ou $t = 33s$), nous observons systématiquement que **la courbe bleue (FPGA) est inférieure à la courbe orange (STM32)**. La courbe verte, qui trace la différence nette entre les deux (*Latence STM32 - Latence FPGA*), met en évidence le **Surcoût du RTOS**, qui oscille en moyenne entre 10 et 20 millisecondes.

Cette différence s'explique par les architectures sous-jacentes :

— **Côté FPGA (Exécution Spatiale et Parallèle) :** La *User Logic* en VHDL est cadencée à 100 MHz (période de 10 ns). Dès que le dernier bit du CRC de la requête est validé par l'UART matériel, la machine à états décode l'ordre, lit les registres internes instantanément, et déclenche la sous-FSM d'émission en quelques coups d'horloge. La latence du FPGA est donc quasi-exclusivement dominée par le temps physique de sérialisation des bits sur le câble à 115200 bauds.

— **Côté STM32 (Exécution Séquentielle et RTOS) :** La réponse du microcontrôleur implique une longue chaîne de dépendances logicielles :

1. L'UART matériel lève une interruption.
2. Le CPU Cortex-M7 sauvegarde son contexte d'exécution pour entrer dans l'ISR `uart_cb`.
3. La FSM logicielle valide le CRC et libère un sémaphore (`k_sem_give`).
4. L'ordonnanceur de Zephyr (Scheduler) effectue un *Context Switch* pour réveiller le *Worker Thread*.
5. Le thread initie une transaction sur le bus I^2C pour interroger la puce PCF8563. Le protocole I^2C standard fonctionnant à **100 kHz**, cette lecture prend intrinsèquement plusieurs millisecondes.
6. Le thread formate la trame de réponse et la renvoie à l'UART.

L'empilement de ces couches d'abstraction (ISR, Scheduler, Driver I^2C) constitue le "coût" de l'approche logicielle mesuré sur notre graphique.

CONCLUSION DU HW/SW CO-DESIGN

La flexibilité face au déterminisme

Cette étude comparative valide les principes fondamentaux de l'ingénierie des systèmes embarqués. L'approche **Hardware (FPGA)** excelle par son **déterminisme absolu** et sa latence minimale. Elle est incontournable pour des applications critiques nécessitant une précision nanoseconde (radars, contrôle moteur haute fréquence).

L'approche **Software (STM32 + Zephyr)**, bien qu'introduisant une latence inévitable (mesurée ici à ∼15 ms de surcoût), offre en contrepartie une **flexibilité de développement inégalée**. Elle nous a permis d'intégrer rapidement un OS temps réel, de piloter un composant industriel externe en I^2C, et de déployer une interface graphique complexe (LVGL) sur un écran tactile, des tâches qui auraient été d'une complexité colossale à implémenter en VHDL pur.

5 Conclusion Générale du Projet

Ce projet de *Hardware/Software Co-Design* s'est avéré être une exploration complète et rigoureuse des deux grands paradigmes de l'ingénierie des systèmes embarqués : la conception matérielle pure (Silicium) et la conception logicielle sous contraintes temps réel (RTOS).

Partis de zéro, nous avons d'abord architecturé un système sur puce (SoC) complet sur FPGA (Nexys A7 100T). Ce développement en VHDL nous a permis de maîtriser les fondamentaux de l'électronique numérique asynchrone (synchroniseurs, suréchantillonnage UART) et synchrone. La conception de la *User Logic* a transformé une simple horloge en un périphérique déterministe, doté d'une sécurité matérielle (*Sanitization*), d'un arbitrage matériel strict, et d'une capacité de notification asynchrone à la nanoseconde près.

Dans un second temps, nous avons basculé dans le monde logiciel en développant un firmware en C sous Zephyr RTOS pour un microcontrôleur industriel (STM32F746G-DISCO). Ce portage a mis en lumière les défis de la programmation concurrente. L'utilisation de sémaphores, de mutex et de files de messages (`k_msgq`) a été indispensable pour sécuriser les transactions entre l'interruption UART, le bus I^2C et l'interface graphique (LVGL), tout en implémentant une tolérance aux pannes inédite via notre horloge virtuelle de secours (*Fallback*).

Enfin, pour orchestrer et arbitrer ce duel technologique, nous avons conçu une application hôte Python asynchrone de qualité industrielle. Véritable centre de contrôle, elle intègre un *Watchdog* de résilience matérielle, un tableau de bord symétrique et un outil métrologique d'acquisition de données (Data Logger).

Le verdict de l'analyse comparative :
La confrontation finale de ces deux architectures face au même protocole de communication a mis en exergue le compromis fondamental de l'ingénierie moderne : **Déterminisme contre Flexibilité**.

- L'approche matérielle (**FPGA**) a brillé par son exécution *Cycle-Accurate* et son absence quasi totale de latence interne, justifiant son usage dans les systèmes critiques (aéronautique, radar, haute fréquence).
- L'approche logicielle (**STM32**), bien qu'imposant un surcoût mesurable d'environ 15 millisecondes dû aux changements de contexte de l'OS et à la lenteur des bus périphériques (I^2C), a prouvé sa supériorité incontestable en matière de flexibilité, permettant l'intégration rapide d'interfaces riches et de protocoles complexes.

En conclusion, ce projet ne se contente pas de répondre à un cahier des charges académique : il démontre une compréhension globale et profonde de l'architecture des systèmes embarqués contemporains. Le choix entre FPGA et microcontrôleur n'est pas une question de supériorité absolue, mais d'adéquation millimétrée aux contraintes du monde réel.